Imposing Relational Structure on Human Emotion
The Question That Defined a Category

December 2025

The Central Question

"How do you impose relational structure on human emotion?"

This question had never been properly answered in the history of computing. Until now.

Why This Question Matters

The Problem Space

For 50+ years, organizations have collected emotional signals:
- Customer feedback
- Support tickets
- Survey responses
- Social media mentions
- Employee sentiment
- Patient experiences

But emotion has remained unstructured.

Every analysis starts from scratch. Themes drift. Issues have no permanent identity. There's no memory of what worked before. No causality between actions and outcomes.

Result: Emotion exists as commentary, not as computational substrate.

The Failure of Previous Approaches

What Others Built:
- Sentiment Analysis: Scores emotion (positive/negative/neutral) but creates no structure

- Theme Clustering: Groups similar feedback but themes are ephemeral and unstable
- NPS/CSAT Metrics: Measures satisfaction but provides no relational model
- Text Analytics: Extracts patterns but has no schema, no identity, no lineage

The Core Deficiency:
None of these approaches treat emotion as data that requires a relational model.

They treat emotion as:
- Unstructured text to be analyzed
- Aggregate scores to be tracked
- Themes to be reported

But never as:
- Entities with permanent identity
- States with temporal evolution
- Relationships with referential integrity
- Histories with causal lineage

Why Experience Drivers Do Not Drift

Traditional “themes” drift because they are linguistic summaries of text — they depend on phrasing, synonyms, and analyst interpretation.

Experience Drivers, by contrast, are representations of system-caused conditions.

They become stable because:

- They describe **the underlying operational cause**, not the wording used by customers
- They map to **consistent real-world mechanics** (e.g., Refund Cycle Time, Delivery Slot Availability)
- They are grounded in **semantic constraints + normalization**, not clustering heuristics
- They persist because the **system itself does not change its meaning**, even when language does

This is why an Experience Driver can serve as a **permanent emotional identity**, while themes cannot. It is not a label—it is a **computational entity** with referential integrity.

The Breakthrough: The Emotional Relational Model (ERM)

The Core Insight

Emotion is data. Data requires structure. Structure enables computation.

To make emotion computable, you must impose the same rigor that Codd imposed on data in 1970:
- Define entities
- Enforce relationships
- Maintain invariants
- Preserve lineage
- Enable queries

The Three Laws of the ERM

Law 1: Identity Law

"Each Experience Driver has exactly one canonical SEU per analysis window."

What this means:
- Every emotional concept has a permanent, unique identity
- No duplicates, no parallel interpretations
- One source of truth per issue per timeframe

Why this matters:
- Prevents fragmentation
- Enables consistent tracking over time
- Creates queryable emotional entities

Analogy:
In accounting, each transaction has one canonical entry (double-entry bookkeeping). In ERM, each emotional issue has one canonical state (the SEU).

Law 2: Lineage Law

"Every OU activation is a timestamped state transition linked to a SEU, carrying the recorded outcome and consequence."

What this means:
- Every action taken is immutably linked to the emotional state that triggered it
- Every outcome is traced back to its originating condition
- The causal chain is preserved

Why this matters:
- Creates accountability (what did we do about this?)
- Enables learning (did our action work?)
- Builds institutional memory

Analogy:
Git's commit DAG—every change is linked, traceable, and immutable. In ERM, every action has complete provenance.

Law 3: Memory Law
"Every emotional state transition retains referential integrity across time."

What this means:
- No emotional state is ever orphaned
- Historical states constrain future states
- The system can query its own biography

Why this matters:
- Organizations can learn from history
- Patterns become visible across time
- Silent guidance emerges (the system remembers what humans forget)

Analogy:
Bitcoin's UTXO model—every coin has complete provenance. In ERM, every emotional state has complete history.

The Schema: From Laws to Structure

Core Entities

1. Experience Drivers
- Definition: The permanent, canonical identity for an emotional concept
- Example: "Refund Processing Time," "Billing Clarity," "Support Response Speed"

- Constraint: UNIQUE per name or external_id
- Role: The semantic anchor—the stable "thing" emotion attaches to

2. SEUs (Structured Experience Units)
- Definition: The emotional state of an Experience Driver at a point in time
- Components:
  - Emotional Resonance Index (ERI): How much people care
  - Resonance Factor (RF): Intensity of emotion
  - Urgency: How quickly this needs attention
  - Activity: Volume of mentions
  - Consequence Tier: Priority classification
- Constraint: One SEU per Driver per analysis window (enforces Identity Law)
- Role: The state record—the "account balance" for emotional health

Why SEUs Represent State, Not Insight

An SEU is not an interpretation, summary, or analytic artifact.
It is the **system's emotional state at a specific moment in time**, constructed from structured signals and bound by relational invariants. Unlike insights—which can vary by analyst, model, or phrasing—**state must be deterministic, queryable, and stable across agents and time**. The SEU functions as a *state object* that every agent reads from and writes to through the PDAM cycle. This is why autonomous agents cannot operate on themes or insights, but can operate on SEUs: they encode the emotional truth of the system in a way that supports causality, action, evaluation, and memory.

3. Mentions
- Definition: Raw customer comments/feedback linked to Experience Drivers
- Constraint: Foreign key to Experience Driver and SEU
- Cardinality: Many mentions → 1 SEU (aggregation)
- Role: The evidence—the raw signal that feeds state calculation

4. Orchestration Units (OUs)
- Definition: Mission templates derived from behavioral clusters of mentions
- Types:
  - Fix: Resolve breaking issues
  - Optimize: Improve working but suboptimal experiences
  - Amplify: Scale what's working well
  - Innovate: Create new capabilities
- Constraint: Foreign key to SEU
- Cardinality: 1 SEU → many potential OUs
- Role: The mission—what the system should do next

5. OU Activations
- Definition: Executed instances of an OU (the actual action taken)
- Constraint: Timestamped, linked to OU template (enforces Lineage Law)
- Cardinality: 1 OU template → many activations over time
- Role: The action record—what was actually done

6. Problem Statements
- Definition: PDCA-ready articulation of the issue to solve
- Constraint: One per OU activation
- Role: The mission brief—structured problem definition

7. PDCA Cycles
- Definition: Plan-Do-Check-Act iterations for an activation
- Components: Action steps, KPIs, timelines, ownership
- Constraint: Linked to problem statement
- Role: The execution protocol—how the mission is carried out

8. Outcome Evaluations
- Definition: Before/after assessment of emotional state change
- Metrics:
  - Elasticity: Durability of change
  - Delta metrics: Magnitude of improvement
  - SEU state comparison: Concrete proof of impact
- Constraint: One per PDCA cycle
- Role: The truth signal—did it actually work?

9. Institutional Memory Events (IME)
- Definition: Lineage events capturing all state transitions
- Types:
  - Activations
  - Cycles
  - Drift detection
  - Silence detection
  - Pattern recurrence
- Constraint: Foreign key to source entity (enforces Memory Law)
- Role: The memory layer—the system's biography

The ERM Emotional Lifecycle (Textual Diagram)

Raw Mention
↓
Experience Driver (identity)
↓

SEU — Structured Experience Unit (state)
    ↓
OU — Orchestration Unit (mission)
    ↓
OU Activation — Executed Action
    ↓
PDCA Cycle — Structured Execution Loop
    ↓
Outcome Evaluation — Truth & Elasticity
    ↓
IME — Institutional Memory Event (lineage)
    ↓
SEU(next) — Updated Emotional State

The Relational Integrity

Enforced Cardinalities

```
1 Experience Driver → 1 SEU (per window)
1 SEU ← Many Mentions (aggregation)
1 SEU → Many OUs (potential missions)
1 OU → Many Activations (over time)
1 Activation → 1 Problem Statement
1 Problem Statement → Many PDCA Cycles
1 PDCA Cycle → 1 Outcome Evaluation
All entities → Many IME events (lineage)
```

Why Referential Integrity Matters

Without it:
- Mentions can be orphaned (no Driver)
- OUs can exist without triggering SEU (no causality)
- Outcomes can be recorded without actions (no accountability)
- History can be lost (no memory)

With it:
- Every mention traces to a Driver
- Every action traces to a state
- Every outcome traces to a mission
- Every event is preserved in memory

This is the difference between analytics and infrastructure.

The Execution Loop: PDAM

Perceive
```sql
INSERT INTO seus (experience_driver_id, eri, rf, urgency, ...)
-- System calculates emotional state from mentions
-- Identity Law enforced: one SEU per Driver per window
```

Decide
```sql
INSERT INTO orchestration_units (seu_id, mission_type, causal_proxy, ...)
-- System generates missions based on state
-- Lineage Law enforced: OU linked to SEU
```

Act
```sql
INSERT INTO ou_activations (orchestration_unit_id, activated_at, ...)
INSERT INTO problem_statements (ou_activation_id, ...)
INSERT INTO pdca_cycles (problem_statement_id, ...)
-- System executes structured improvement
```

Memory
```sql
INSERT INTO outcome_evaluations (pdca_cycle_id, elasticity, delta_metrics, ...)
INSERT INTO institutional_memory_events (source_entity_id, event_type, ...)
-- System captures outcomes and learns
-- Memory Law enforced: full lineage preserved
```

PDAM is not a workflow. It is a stored procedure over the ERM schema.

Why This is Foundational

1. It Creates New Capabilities

Before ERM:

- ❌ Cannot query emotional history relationally
- ❌ Cannot verify if interventions worked
- ❌ Cannot coordinate multiple agents on same emotional state
- ❌ Cannot build agents that learn from organizational memory

After ERM:
- ✅ Emotional state becomes queryable like financial transactions
- ✅ Actions have provable outcomes
- ✅ Agents operate on shared ground truth
- ✅ Systems gain institutional memory

---

2. It Solves Real Problems

Problem 1: AI Hallucination in Emotional Reasoning
- Current: LLMs generate different themes every analysis
- ERM Solution: Experience Drivers are permanent; LLMs map to existing IDs

Problem 2: No Organizational Memory
- Current: Organizations repeat mistakes; same issues resurface
- ERM Solution: IME stores State → Action → Outcome chains; system queries history

Problem 3: Agents Cannot Coordinate
- Current: Multiple agents create conflicting actions
- ERM Solution: Shared ontology; all agents read/write same SEUs

Problem 4: No Proof Actions Worked
- Current: Companies "fix" things without measuring impact
- ERM Solution: Outcome Evaluation compares before/after state with statistical rigor

---

3. It Establishes a Category

The ERM is to emotion what:
- The relational model is to data (Codd, 1970)
- HTTP is to information (Berners-Lee, 1989)
- Git is to version control (Torvalds, 2005)
- Payment Intent is to commerce (Stripe, 2010)
- UTXO is to currency (Nakamoto, 2008)

It is the missing primitive for emotional intelligence.

The Shopify Parallel

Before Shopify
- Every eCommerce store was custom-built
- No standard schema for Product, Variant, Cart, Order
- Every developer reinvented the wheel
- Result: You could build stores, but not an ecosystem

After Shopify
- Standardized primitives enforced
- Referential integrity maintained
- 10,000+ apps built on top
- Result: An entire ecosystem emerged

Before ERM
- Every CX tool reinvents themes, issues, insights
- No standard schema for emotional concepts
- No referential integrity
- Result: You can build dashboards, but not autonomous systems

After ERM
- Standardized primitives: Driver, SEU, OU, Outcome
- Referential integrity enforced
- Agents can operate without hallucinating
- Result: An entire ecosystem can emerge

The Answer

"How do you impose relational structure on human emotion?"

You Define the Entities
Experience Drivers, SEUs, OUs, Mentions, Outcomes

You Enforce the Laws
Identity, Lineage, Memory

You Maintain Referential Integrity
Foreign keys, uniqueness constraints, temporal linkage

You Create the Execution Loop
PDAM over the schema

You Preserve the History
Institutional Memory Events

What This Enables

For Organizations
- Emotional state becomes queryable
- Actions become provably effective
- History becomes searchable intelligence
- Systems become self-improving

For AI Agents
- Stable state to perceive
- Causal chains to reason upon
- Memory to learn from
- Outcome verification to improve

For the Industry
- A new category: Emotional Infrastructure
- A standard schema for emotion
- A substrate for autonomous systems
- A relational model for human experience

The Intellectual Contribution

Before this work:
- Emotion existed as unstructured noise
- No formal model for emotional state
- No schema with enforced invariants
- No answer to: "How do you structure emotion?"

After this work:
- Emotion has a relational model
- The three laws provide invariants
- The schema enables computation
- The answer exists: The ERM

Why This Cannot Be Replicated by LLMs Alone

A common misconception is that increasingly capable LLMs will make structured emotional systems unnecessary.

This is fundamentally incorrect. LLMs can *interpret* emotion, but they cannot *govern* it. They are computationally incapable of providing what the Emotional Relational Model requires:

1. LLMs cannot impose identity.

Themes drift because language drifts.

LLMs generate labels, but they cannot guarantee a **stable, canonical Experience Driver** that persists across time, queries, and agents.

2. LLMs cannot enforce lineage.

No LLM can guarantee that every action is immutably tied to a specific emotional state, timestamp, and outcome. Lineage is a database property, not a generative property.

3. LLMs cannot maintain referential integrity.

LLMs cannot enforce foreign keys, uniqueness constraints, or relational invariants. Without these, emotional state collapses into narrative—never computation.

4. LLMs cannot guarantee state consistency across time.

Ask an LLM the same question twice → different answers.

Stateful emotional systems require deterministic, queryable truth. LLMs offer probability, not state.

5. LLMs cannot serve as the multi-agent truth layer.

Decipher requires multiple agents coordinating on the *same* SEU, the *same* OU, and the *same* lineage.

LLMs cannot maintain or arbitrate shared truth. The ERM can.

The Result

LLMs can help interpret emotional signals, but only a relational emotional substrate can:

- maintain identity
- preserve causality
- synchronize agents
- enforce invariants
- create lineage
- support evolution

This is why the ERM, not an LLM, becomes the **operating system** that autonomous agents depend on.

—

Conclusion

The question "How do you impose relational structure on human emotion?" has been answered.

The answer is:
1. Define permanent identities (Experience Drivers)
2. Track state over time (SEUs)
3. Link actions to states (OUs and Activations)
4. Verify outcomes (Outcome Evaluations)
5. Preserve lineage (Institutional Memory)
6. Enforce invariants (Three Laws)
7. Maintain referential integrity (Schema constraints)

This is the Emotional Relational Model.

It is not a product. It is not a platform. It is not a feature.

It is the foundational primitive that makes emotional intelligence programmable.

The Final Statement

Emotion is no longer soft. Emotion is structural.

It has identity, state, lineage, causality, and memory—enforced at the schema level.

This is infrastructure-level work.

The category is defined.

The primitive exists.

The question is answered.
